

ASSIGNMENT 2

DESIGN OF 4 BITS RIPPLE CARRY ADDER

SUBHAZRA

SUBHRADEEP HAZRA
subhradeephazra5240@gmail.com

Full Adder

Code:

```
module full_adder (  
    input A,  
    input B,  
    input Cin,  
    output Sum,  
    output Cout  
);  
  
    wire x1, c1, c2;  
  
    // xor for sum  
    xor (x1, A, B);  
    xor (Sum, x1, Cin);  
  
    // carry logic  
    and (c1, A, B);  
    and (c2, x1, Cin);  
    or (Cout, c1, c2);  
  
endmodule
```

Ripple Carry Adder

Code:

```
module ripple_adder_4bit (  
    input [3:0] A,  
    input [3:0] B,  
    input Cin,  
    output [3:0] Sum,  
    output Cout  
);  
  
    wire c1, c2, c3;  
  
    // instantiate 4 full adders  
    full_adder fa0 (A[0], B[0], Cin, Sum[0], c1);  
    full_adder fa1 (A[1], B[1], c1, Sum[1], c2);  
    full_adder fa2 (A[2], B[2], c2, Sum[2], c3);  
    full_adder fa3 (A[3], B[3], c3, Sum[3], Cout);  
  
endmodule
```

Test Bench

Code:

```
`timescale 1ns/1ps

module testbench_4bitadder;

    reg [3:0] A, B;
    reg Cin;
    wire [3:0] Sum;
    wire Cout;

    // instantiate the dut
    ripple_adder_4bit uut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .Sum(Sum),
        .Cout(Cout)
    );

    initial begin
        A=4'b0000; B=4'b0000; Cin=0; #10;
        A=4'b0101; B=4'b0011; Cin=0; #10;
        A=4'b1111; B=4'b0001; Cin=0; #10;
        A=4'b1010; B=4'b0101; Cin=1; #10;
        A=4'b1111; B=4'b1111; Cin=1; #10;

        $finish;
    end

endmodule
```

Result

Simulation:

 /testbench_4bitAdd...	0000	0000	0101	1111	1010	1111	
 /testbench_4bitAdd...	0000	0000	0011	0001	0101	1111	
 /testbench_4bitAdd...	St0						
 /testbench_4bitAdd...	0000	0000	1000	0000	0000	1111	
 /testbench_4bitAdd...	St0						
 /testbench_4bitAdd...	St0						

Critical Path:

Cin → fa0 → fa1 → fa2 → fa3 → Cout